

Letný report

Práca vykonaná počas semestra:

- Projektu z prvého semestra bol ďalej rozvíjaný predtým, ako boli v algoritme objavené kritické nedostatky, ktoré si vyžiadali zjednodušenie kódovej základne.
- SAT a ILP solvery boli pôvodne považované za alternatívne riešenie, ale boli opustené, keď sa našli efektívnejšie algoritmické riešenia.
- Formulácie pre SAT a ILP sa ukázali ako užitočné, pretože objasnili skutočnosť, že multipóly sa dajú generovať sekvenčne simuláciou vyhľadávania do šírky a že rekurzívny kód sa dá jednoduchšie implementovať pre pevný počet vrcholov.
- Relatívna účinnosť vyhľadávania do hĺbky a vyhľadávania do šírky bola hodnotená z hľadiska ich schopnosti generovať $(3,g,s)$ -multipóly. DFS bol nakoniec vylúčený kvôli požiadavke na zložitejšie backtracking. Pomocou BFS bolo možné navštíviť každý vrchol iba raz pre každý navrhovaný multipól.
- Bol identifikovaný algoritmus na rýchle určenie obvodu grafu.
- Boli preskúvané kanonické formy grafov a boli zvážené metódy na elimináciu izomorfizmu.

Štruktúra projektu:

- Keďže cieľom projektu je generovanie $(3,g,s)$ -multipólov, kód ovládača berie ako vstup g a s . Potom začne generovať multipóly od najmenšieho
- Pre každý cieľový počet vrcholov kód ovládača volá metódu **start()** (niektoré nemožné počty vrcholov preskočí)
- Funkcia **start()** inicializuje premenné. Potom volá metódu **assign_semiedges()** s počiatočnými parametrami (alebo priamo **recursion()**, ak s je 0).
- Funkcia **assign_semiedges()** sa volá rekurzívne na priradenie počtu polohran. Je optimalizovaná tak, aby vrcholy s polohranami tvorili súvislý segment na začiatku zoznamu susednosti, pričom vrcholy s väčším počtom polohran sú bližšie k začiatku. Tým sa eliminuje všetok izomorfizmus spôsobený umiestnením polohran.
- Funkcia **recursion()**: Základnou myšlienkou rekurzívneho kódu je simulovať BFS strom pre každý možný $(3,g,s)$ -multipól veľkosti N . Stále je prítomný nezanedbateľný počet izomorfných multipólov, ale prevažná väčšina z nich sa nikdy nevygeneruje.
- Počas každého rekurzívneho kroku sa vyberie ďalší voľný pahýľ (nepriradená hrana, po anglicky *stub*). Vždy sa vyberajú postupne, zľava doprava v zozname susednosti.

- Pre účely tohto vysvetlenia sa vrchol v považuje za rodiča vrcholu u , ak existuje hrana $\{v, u\}$, kde $v < u$. Podobne sa u považuje za potomka vrcholu v .
- Po vybratí pahýľa sa vyskúšajú nasledujúce možnosti:
 1. Nahradiť pahýľ hranou pre potenciálneho potomka, ktorý už má rodiča alebo polohranu.
 2. Nahradiť pahýľ hranou k potomkovi, ktorý má zatiaľ iba pahýle. (iba raz, k vrcholu s najmenším možným takýmto indexom – ďalšie pokusy by vytvorili iba izomorfné grafy)
- Po každej úspešnej možnosti sa kontroluje, či je obvod stále väčší alebo rovný g . Ak áno, nasleduje rekúzia na ďalšej hrane, potom backtracking.
- Akonáhle nezostanú žiadne voľné výčnelky, multipól je $(3,g,s)$ -multipól a vytlačí sa na konzolu. Potom sa program ide ďalej.
- Kód poskytuje výsledky slušným tempom až do približne 40 vrcholov, ale má potenciál na ďalšiu optimalizáciu. Značnú réžiu predstavuje väčší obvod a izomorfizmus pri vyššom počte vrcholov.

Možné optimalizácie:

- Sledovanie voľných pahýľov pre rýchlejší prístup
- Implementácia kontroly kanoničnosti a odstraňovania nekanonických multipólov.
- Optimalizácia určenie obvodu – tá istá operácia sa opakovane vykonáva na multipóle, ktorý sa mení len veľmi málo. Možno by sa niektoré zhromaždené informácie mohli použiť (napr. zapamätanie si minimálnych vzdialeností až do $g - 1$)
- Redukcia symetrických multipólov: Ak dva vrcholy zdieľajú jedného alebo dvoch rodičov (sú to „súrodenci“), druhý súrodenec môže mať iba potomkov s vyšším indexom ako prvý.